

Prof-Finder：基于 LLM 与语义匹配的导师推荐系统

技术选型与产品体系结构设计报告

1 技术选型

1.1 选型原则

技术选型遵循以下原则：

1. **轻量化优先**：选择轻量级框架与嵌入式数据库，降低部署与运维复杂度。
2. **Python 生态优先**：核心业务逻辑集中在 Python 生态，保证机器学习、大模型等库的兼容性。
3. **现代前端工程化**：前端采用 TypeScript + 组合式 API + 原子化 CSS。
4. **异步优先**：后端全链路支持 async/await，避免 I/O 密集型操作阻塞请求。
5. **跨平台兼容**：开发和使用都同时支持到 macOS 与 Windows，照顾到组内的所有成员。

1.2 技术选型说明

1.2.1 后端

FastAPI 作为 Web 框架，天然支持 async/await，适合处理 I/O 密集型操作（LLM 调用、爬虫请求、文件解析）；内置 Pydantic 数据验证与 OpenAPI 文档自动生成。

SQLAlchemy 2.0 + SQLite 作为持久层。SQLAlchemy 提供成熟 ORM；SQLite 免去独立数据库服务的运维开销。后期可无缝迁移至 PostgreSQL。

sentence-transformers 加载 allennlp-specter 模型（基于 SciBERT 微调），将教授研究方向与学生简历映射至同一语义空间，计算余弦相似度。

OpenAI SDK 调用 DeepSeek API（OpenAI 兼容接口），完成简历解析、论文摘要与邮件生成。

Huey 作为轻量级任务队列，配合 SQLite 实现后台任务的可靠调度。

scholarly + BeautifulSoup4 + requests 构成爬虫工具链：scholarly 负责 Google Scholar，BeautifulSoup4 负责大学院系网页解析。

pymupdf4llm 将 PDF 论文转换为 Markdown，供 LLM 进行摘要与关键词提取。

1.2.2 前端

Vue 3 + Composition API + TypeScript 为核心框架，提供灵活的逻辑复用与静态类型检查。

Vite 利用原生 ES 模块实现极速冷启动与 HMR。

表 1: 系统开发环境 (已在 macOS 26 Tahoe 完成环境搭建)

开发环境	环境参数
开发操作系统	已在 macOS 26 Tahoe 完成环境搭建
内存	16GB
CPU	M2 (ARM64)
Python 环境管理	Conda (prof-finder 环境)
Python 版本	3.11.15
Python 包管理器	Poetry 2.3.4
后端框架	FastAPI 0.109.2
ASGI 服务器	Uvicorn 0.27.1
ORM 框架	SQLAlchemy 2.0.46
数据库	SQLite
任务队列	Huey 3.0.0
语义嵌入框架	sentence-transformers 5.2.3
语义嵌入模型	allenai-specter
LLM SDK	OpenAI SDK 1.109.1
爬虫库	scholarly 1.7.11 / requests 2.32.5 / BeautifulSoup4 4.14.3
PDF 解析	pymupdf4llm 0.3.4
前端框架	Vue 3.5.24
TypeScript 版本	5.9.3
构建工具	Vite 7.3.1
UI 组件库	Naive UI 2.43.2
CSS 框架	Tailwind CSS 4.2.4
状态管理	Pinia 3.0.4
HTTP 客户端	Axios 1.13.3
实时通信	EventSource (SSE) / sse-starlette 2.4.1
Node.js 版本	v24.15.0
npm 版本	11.12.1
IDE	VSCode / PyCharm
版本控制	Git 2.46.1
代码格式化	Black 23.12.1
代码检查	Flake8 6.1.0 / MyPy 1.19.1
测试框架	pytest 7.4.4

表 2: 系统开发环境 (已在 Windows 11 完成环境搭建)

开发环境	环境参数
开发操作系统	已在 Windows 11 完成环境搭建
内存	16GB
CPU	Intel i7-12700H
Python 环境管理	Conda (prof-finder 环境)
Python 版本	3.11.15
Python 包管理器	Poetry 2.3.4
后端框架	FastAPI 0.109.2
ASGI 服务器	Uvicorn 0.27.1
ORM 框架	SQLAlchemy 2.0.46
数据库	SQLite
任务队列	Huey 3.0.0
语义嵌入框架	sentence-transformers 5.2.3
语义嵌入模型	allenai-specter
LLM SDK	OpenAI SDK 1.109.1
爬虫库	scholarly 1.7.11 / requests 2.32.5 / BeautifulSoup4 4.14.3
PDF 解析	pymupdf4llm 0.3.4
前端框架	Vue 3.5.24
TypeScript 版本	5.9.3
构建工具	Vite 7.3.1
UI 组件库	Naive UI 2.43.2
CSS 框架	Tailwind CSS 4.2.4
状态管理	Pinia 3.0.4
HTTP 客户端	Axios 1.13.3
实时通信	EventSource (SSE) / sse-starlette 2.4.1
Node.js 版本	v24.15.0
npm 版本	11.12.1
IDE	VS Code / PyCharm
版本控制	Git 2.46.1
代码格式化	Black 23.12.1
代码检查	Flake8 6.1.0 / MyPy 1.19.1
测试框架	pytest 7.4.4

Naive UI 提供 90+ 组件，天然 Tree Shaking 与 TypeScript 支持。
Tailwind CSS 以原子化方式构建样式，减少手写 CSS。
Pinia 作为 Vuex 继任者管理全局状态，提供简洁 API 与 TypeScript 推导。
SSE 接收后端异步任务进度推送，比 WebSocket 更轻量，符合单向推送场景。

2 产品体系结构设计

2.1 整体架构

系统采用**前后端分离**的经典分层架构，共分为四层：

层次	职责	关键技术
表示层 (Presentation)	用户界面、交互逻辑、路由	Vue 3 + Naive UI + Tailwind
服务层 (Application)	API 网关、认证、业务编排	FastAPI + JWT + SSE
领域层 (Domain)	爬虫、简历解析、语义匹配、LLM 调用	scholarly + sentence-transformers + OpenAI SDK
持久层 (Persistence)	数据存储、缓存、迁移	SQLAlchemy + SQLite

前后端通过 HTTP/HTTPS 协议通信，API 统一以 `/api` 为前缀。前端构建产物为纯静态文件 (HTML/CSS/JS)，可由任意 Web 服务器 (Nginx、Caddy) 或 CDN 托管。后端作为独立进程运行，通过 ASGI 服务器 (Uvicorn) 对外暴露接口。

浏览器	→	Nginx	→	静态资源 (dist/)
→ API 服务 (Uvicorn:8000)				

2.2 前端体系结构

2.2.1 路由设计

采用 Vue Router 的 history 模式，路由映射如下：

路由路径	页面	权限
/login	登录	公开
/register	注册	公开
/change-password	修改密码	已登录 (首登强制跳转)
/profile	简历管理	已登录
/professor	教授管理	已登录
/professor/:id/edit	教授信息编辑	已登录
/match	匹配结果	已登录
/letter	邮件管理	已登录
/settings	用户设置	已登录
/admin/users	用户管理	仅管理员

2.2.2 实时通信

异步任务（爬取、匹配、LLM 调用）通过 SSE 实现进度推送：

1. 用户在前端发起操作（如”爬取教授”）
2. 前端调用 API 获取 `task_id`，立即返回
3. 前端创建 `EventSource` 连接到 `/api/tasks/{task_id}/progress`
4. 后端异步执行任务，通过 SSE 推送进度事件（`progress` / `complete` / `failed`）
5. 前端 `TaskPanel` 组件实时更新进度条与状态

2.3 后端体系结构

2.3.1 分层设计

后端采用**四层架构**组织代码：

层次	目录	职责
API 层	<code>api/</code>	路由定义、请求验证、依赖注入、SSE 端点
业务层	<code>crawler/</code> , <code>matcher/</code> , <code>llm/</code> , <code>parser/</code>	爬虫调度、语义匹配、LLM 调用、简历解析
数据层	<code>db/</code> , <code>models/</code>	数据库连接、ORM 模型、CRUD 操作
配置层	<code>prompts/</code>	YAML Prompt 模板、配置常量

2.3.2 API 路由规划

所有路由统一以 `/api` 为前缀，按资源类型划分：

路由前缀	资源	核心端点
<code>/api/auth/</code>	认证	<code>login</code> , <code>register</code> , <code>refresh</code> , <code>me</code>
<code>/api/profiles/</code>	简历	上传、解析预览、CRUD、激活
<code>/api/professors/</code>	教授	Scholar 爬取、大学批量爬取、CRUD、信息编辑
<code>/api/match/</code>	匹配	运行匹配、获取结果
<code>/api/letters/</code>	邮件	生成、查看、编辑
<code>/api/tasks/</code>	异步任务	SSE 进度、取消、批量清理
<code>/api/source-inputs/</code>	文档输入	PDF 上传、ArXiv 提交、重试
<code>/api/settings/</code>	用户设置	读取、更新 API Key 等

2.3.3 异步任务系统

系统中爬取、LLM 调用、匹配计算均为耗时操作，设计异步任务系统如下：

- **任务管理器**：维护全局任务状态字典，每个任务包含唯一 ID、类型、进度、状态与事件队列
- **任务创建**：API 端点接收请求后，立即创建 `TaskState` 并返回 `task_id`
- **后台执行**：通过 `asyncio.create_task` 启动协程，阻塞 I/O（如 scholarly 请求）通过 `asyncio.to_thread` 卸载到线程池
- **进度推送**：每个任务维护一个 `asyncio.Queue`，SSE 端点从中读取事件并推送到前端
- **取消支持**：任务循环中检查 `cancel_requested` 标志，支持用户主动取消

支持的任务类型包括：单教授爬取、批量爬取、大学院系爬取、语义匹配、单封邮件生成、批量邮件生成、论文摘要。

2.4 数据体系结构

2.4.1 核心数据模型

我们设计了六类核心实体，其关系如图 3 所示。

实体	核心字段	关联
User	id, username, password_hash, is_admin	→ UserSettings, UserProfile, Professor
UserSettings	user_id, deepseek_api_key, request_delay	→ User（一对一）
UserProfile	user_id, title, name, education, skills, is_active	→ User（多对一） → MatchRecord
Professor	user_id, name, affiliation, research_interests, publications, embedding, h_index	→ User（多对一） → MatchRecord, SourceInput
MatchRecord	user_profile_id, professor_id, score, letter_content	→ UserProfile, Professor
SourceInput	source_type, canonical_id, extracted_markdown, status	→ Professor（可选）

表 3: 核心数据模型及其关联

2.4.2 爬虫模块设计

爬虫模块采用**注册表模式**设计，支持可扩展的多数据源接入：

数据源	技术方案	输出
Google Scholar	scholarly 库，按 ID 搜索并填充基本信息、论文列表与引用指标	Professor 结构化字典
大学院系网站	BeautifulSoup4 + requests, 继承 UniversityCrawlerBase 基类	Professor 列表字典

新增大学爬虫只需：实现 `UniversityCrawlerBase` 子类 → 在注册表中添加一行映射。前端通过 `GET /api/professors/university-crawlers` 动态获取可用爬虫列表，实现**开闭原则**（对扩展开放，对修改关闭）。

3 小组成员开发环境构建情况

全体成员已按照第 1、2 节的配置完成开发环境搭建。小组分工与负责方向如下：

姓名	角色	开发环境搭建情况
郝飞洋	项目发起人	已在 macOS 26 Tahoe 完成环境搭建
李晨璞	项目经理	已在 Windows 11 完成环境搭建
谢寒迪	后端架构师	已在 Windows 11 完成环境搭建
李仁昌	数据库工程师	已在 macOS 26 Tahoe 完成环境搭建
白雨欣	前端开发负责人	已在 Windows 11 完成环境搭建
彭子国	AI 与算法工程师	已在 Windows 11 完成环境搭建
李昊文	数据工程师	已在 Windows 11 完成环境搭建
银睿迪	测试 & 文档 & 部署	已在 Windows 11 完成环境搭建

各成员开发环境均已验证通过，项目具备进入下一阶段的条件。